

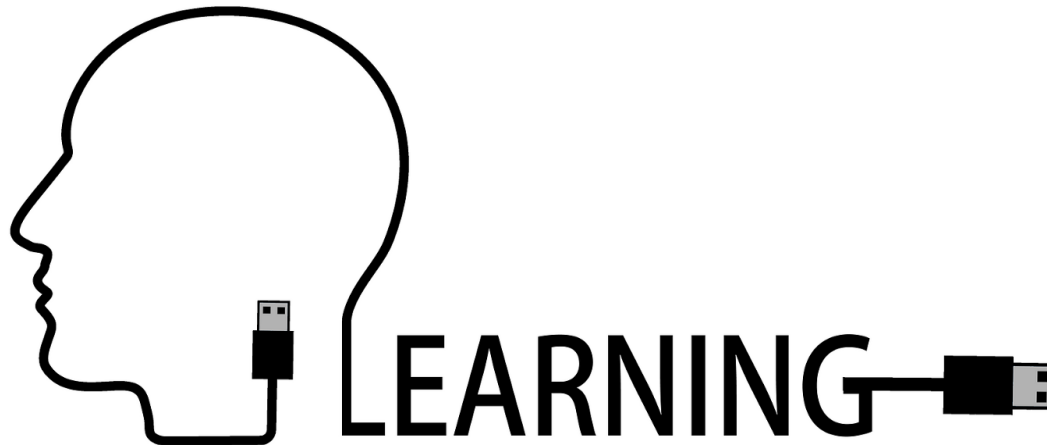
Introduction to Large Language Models (LLMs)

Language Model Training, Finetuning and Inference
Chapter 3

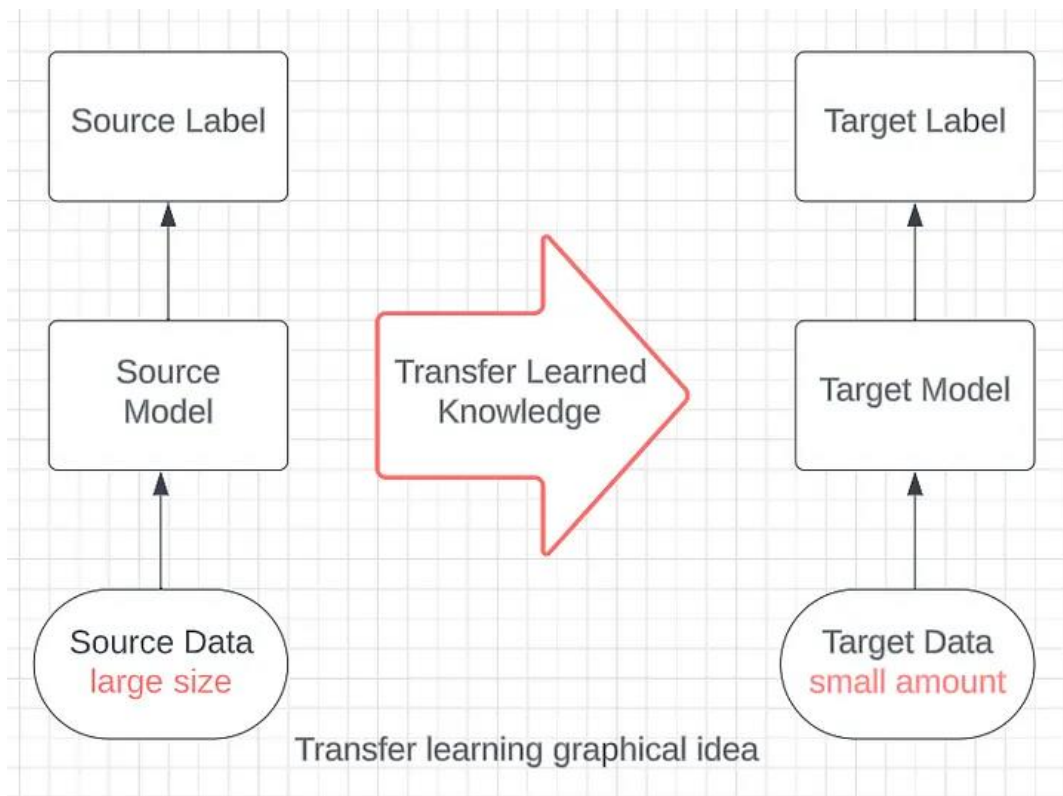


Transfer Learning

- Transfer learning is an approach to machine learning where a model trained on one task is used as the starting point for a model on a new task. This is done by transferring the knowledge that the first model has learned about the features of the data to the second model.



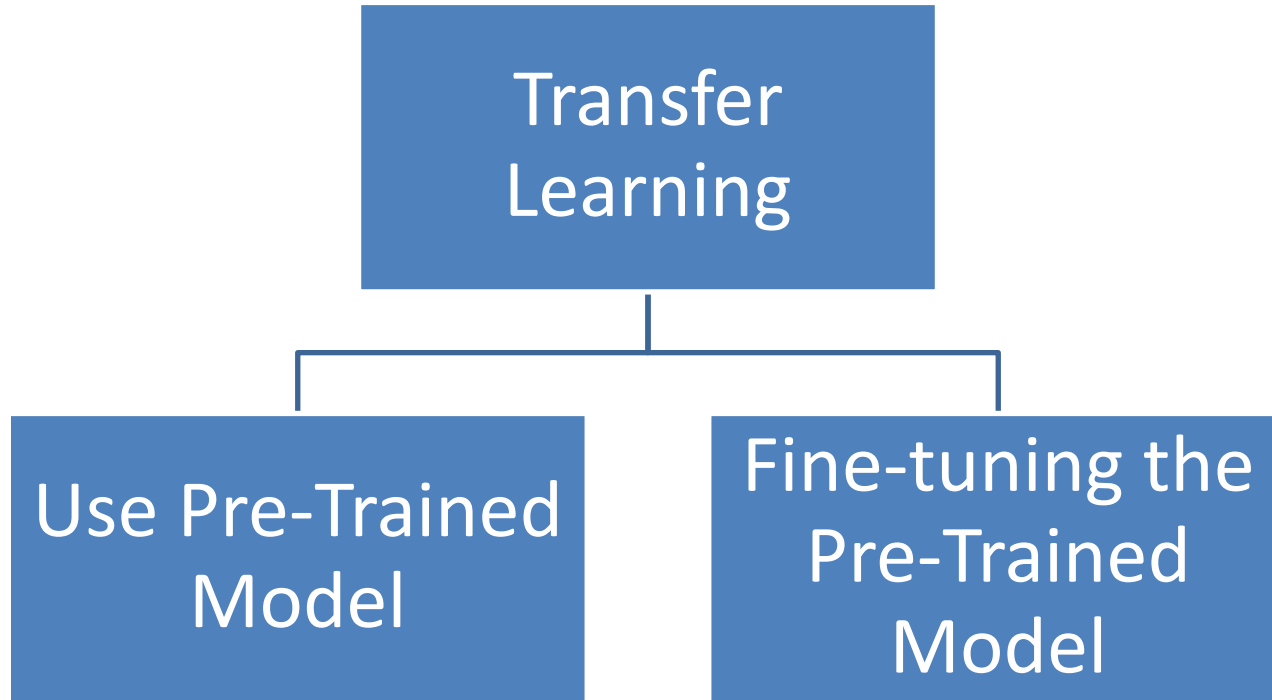
Transfer Learning



@davidfagb



Transfer Learning



Transfer Learning

Feature extraction: In feature extraction, a pre-trained model is leveraged to generate feature representations from the data.

These extracted features are then used to train a separate model for the target task, making it an effective approach when the available data for the task is limited.



Transfer Learning

Fine-tuning: Fine-tuning is a technique where a pre-trained model is further adapted by training it on a new dataset to enhance its performance for a specific task.

While the pre-trained model is originally trained on a large, general dataset, the fine-tuning process uses a smaller, task-specific dataset to tailor the model to the desired application.



What is Transferable Knowledge?

- **Definition:** Knowledge, patterns, and representations learned from one task/domain applied to another.
- **Low-level Features:** Basic, reusable elements (edges, textures, word embeddings).
- **High-level Semantics:** Abstract concepts (objects, scenes, sentiment, intent).



Knowledge Types & Task/Domain Similarity

- **Domain-specific Knowledge:** Task-focused (e.g., medical anatomy).
- **Generic Knowledge:** Broadly useful (syntax, textures, spatial relations).
- **Task Similarity:**
 - High: Related objectives (dog vs. cat classification).
 - Low: Different goals (sentiment analysis vs. landmark detection).
- **Domain Similarity:**
 - High: Similar data (X-rays across hospitals).
 - Low: Different data (X-rays vs. satellite images).



Transfer Techniques

- **Feature Extraction:** Use pre-trained model as a fixed feature extractor → new classifier trains on extracted features.
- **Fine-tuning:** Continue training pre-trained model on new data → adapts to specific task.
- **Same-domain Transfer:** Easier, better performance (similar data).
- **Cross-domain Transfer:** Harder, needs adaptation (different data).



ELMo

Deep Contextualized Word Representations

ELMo, by Allen Institute for Artificial Intelligence, and University of Washington

2018 NAACL, Over 8000 Citations



The "One-Word, One-Vector" Limitation

- Previous popular methods like **Word2Vec** and **GloVe** created a global dictionary of words. Each word in this dictionary was assigned a single, fixed vector representation.
- **Example:** The word "stick" would have *one* vector.
 - "Let's **stick** together." (to remain)
 - "He poked the fire with a **stick**." (a piece of wood)
- A single vector simply can't capture these different meanings. This problem is called **polysemy** (a word having multiple meanings), and it was a major limitation of static word embeddings.



ELMo: Embeddings from Language Models

- ELMo's revolutionary idea was: **Don't use a fixed dictionary. Instead, generate the embedding on the fly using the entire sentence as context.**
- By using a **Language Model (LM)**. A language model is simply a model trained to predict the next word in a sequence. By training on a massive amount of text, the LM learns deep relationships between words, syntax, and semantics.



ELMo

- ELMo uses a specific type of LM: a **deep Bidirectional Language Model (biLM)**. Let's break that down:
- **Bidirectional:** It doesn't just predict the next word (left-to-right). It also has a second model that predicts the *previous* word (right-to-left). This is crucial because a word's meaning is influenced by what comes **before and after** it.
- **Deep:** It's not just one layer. The model consists of multiple layers of LSTMs stacked on top of each other. This is important because different layers capture different types of information.



Simple Bidirectional Language Model

Given a sequence of N tokens, $(t_1, t_2, \dots, t_N)$, a **forward language model** computes the probability of the sequence by modeling the probability of token t_k given the history $(t_1, \dots, t_{k-1})$:

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k \mid t_1, t_2, \dots, t_{k-1}).$$



Simple Bidirectional Language Model

At each position k , each LSTM layer outputs a context-dependent representation:

$$\vec{h}_{k,j}^{LM}$$

where $j=1, \dots, L$.

The top layer LSTM output:

$$\vec{h}_{k,L}^{LM}$$

is used to predict the next token t_{k+1} with a Softmax layer.



Simple Bidirectional Language Model

Similarly, a **backward LM**:

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k \mid t_{k+1}, t_{k+2}, \dots, t_N).$$

A **biLM** combines both a forward and backward LM. The formulation **jointly maximizes the log likelihood of the forward and backward directions**:

$$\sum_{k=1}^N (\log p(t_k \mid t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{LSTM}, \Theta_s) \\ + \log p(t_k \mid t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s)).$$



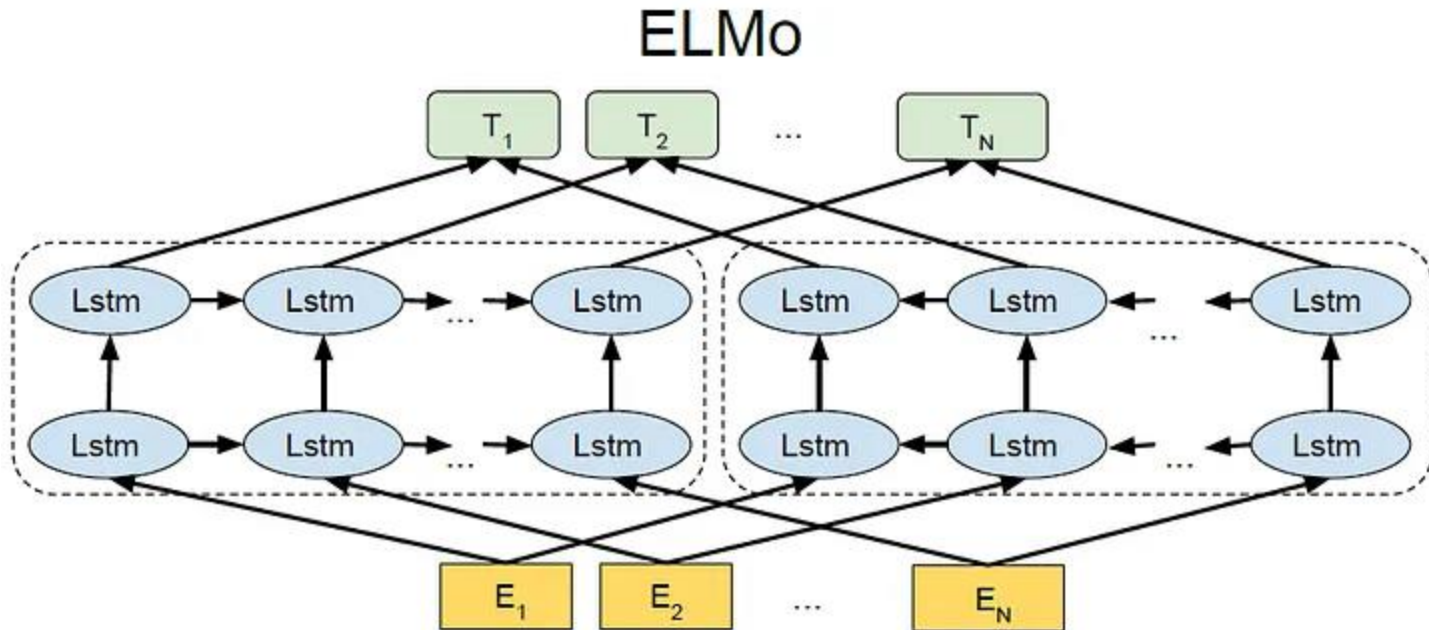
Simple Bidirectional Language Model

In a bidirectional LSTM, the networks processing the forward and backward directions generally maintain their own separate sets of parameters.

However, in some configurations, specific weights may be shared between the two directions rather than keeping completely independent parameters for each.



ELMo



ELMo

- **Step 1: Character-Based Input**
- Instead of looking up a word in a vocabulary, ELMo starts with the characters of the word. It uses a Character-level Convolutional Neural Network (CharCNN) to create a rough, initial embedding.
- **Benefit:** This allows ELMo to handle **out-of-vocabulary (OOV)** words. If it sees a new word like "Jedi," it can still create a reasonable vector based on its characters, rather than just marking it as "unknown."



ELMo

- **Step 2: The Deep Bidirectional LSTM**
- This is the heart of the model. The character-based embedding for each word is fed into a multi-layered biLSTM.
- **Forward LM:** One stack of LSTMs processes the sentence from left to right (e.g., "Let's go to the bank").
- **Backward LM:** A separate stack of LSTMs processes the sentence from right to left (e.g., "bank the to go Let's").



ELMo

- The final ELMo embedding for a word is not just the output of the top layer.
- Instead, it's a **weighted sum** of all the representations from all the layers for that word.



ELMo

The final ELMo vector is calculated as:

$$\text{ELMo}_k = \gamma(s_0 h_{k,0} + s_1 h_{k,1} + s_2 h_{k,2})$$

Where:

- $h_{k,0}$ is the character-based layer, and $h_{k,1}, h_{k,2}$ are the biLSTM layers.
- s_0, s_1, s_2 are **task-specific weights** that are learned during the training of the downstream model (e.g., a sentiment classifier).
- This is a brilliant feature! The downstream model learns how much syntactic vs. semantic information it needs.
 - **Lower layers** tend to capture syntax (part-of-speech, word structure).
 - **Higher layers** tend to capture semantics (context-dependent meaning).
- Gamma is a global scaling factor, also learned for the specific task.



Why ELMo Was Groundbreaking ?

- **Context is King:** It was the first widely successful model to produce truly contextual embeddings, effectively solving the polysemy problem.
- **Deep Representations:** It showed the power of using information from *all* layers of a deep network, not just the final one.
- **Character-Awareness:** Its character-based input made it robust to rare and unknown words.
- **Transfer Learning Powerhouse:** It established a powerful paradigm: pre-train a massive language model on general text, and then use its representations to significantly boost performance on specific downstream tasks (like sentiment analysis, question answering, etc.) with much less task-specific data.



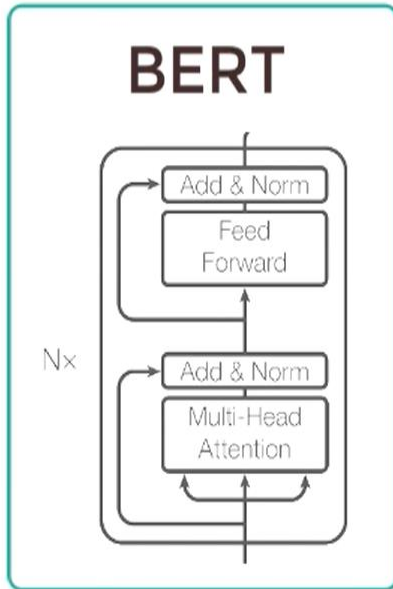
Limitation of ELMo

- ELMo was powerful, but its bidirectionality was "shallow." It ran two separate LSTM models—one forward and one backward—and simply concatenated their results.
- The left-to-right model didn't know anything about the right-to-left context *while it was processing*.
- BERT's goal was to achieve **deep bidirectionality**, where the model could learn from the entire context (both left and right) simultaneously, at every single layer of the network.



Bidirectional Encoder Representation from Transformers

Problems to Solve



- Neural Machine Translation
- Question Answering
- Sentiment Analysis
- Text summarization

Needs Language understanding

How to solve Problems

- Pretrain BERT to understand language
- Fine tune BERT to learn specific task



Bidirectional Encoder Representation from Transformers

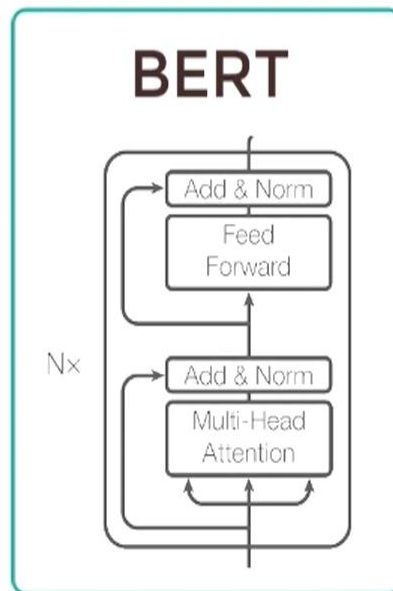
Pretraining (Pass 1) : “What is language? What is context?”

Masked Language
Model (MLM)

The [MASK1] brown
fox [MASK2] over
the lazy dog.

Next Sentence
Prediction (NSP)

A: Ajay is a cool dude.
B: He lives in Ohio



[MASK1] = quick
[MASK2] = jumped

Yes. Sentence B
follows sentence A



Bidirectional Encoder Representation from Transformers

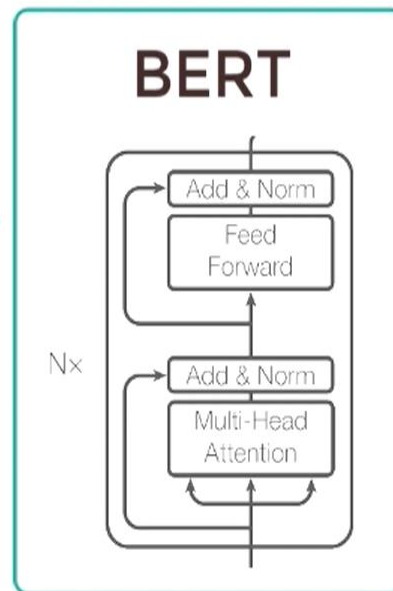
Pretraining (Pass 1) : “What is language? What is context?”

Masked Language
Model (MLM)

The [MASK1] brown
fox [MASK2] over
the lazy dog.

Next Sentence
Prediction (NSP)

A: Ajay is a cool dude.
B: He lives in Ohio



[MASK1] = quick
[MASK2] = jumped

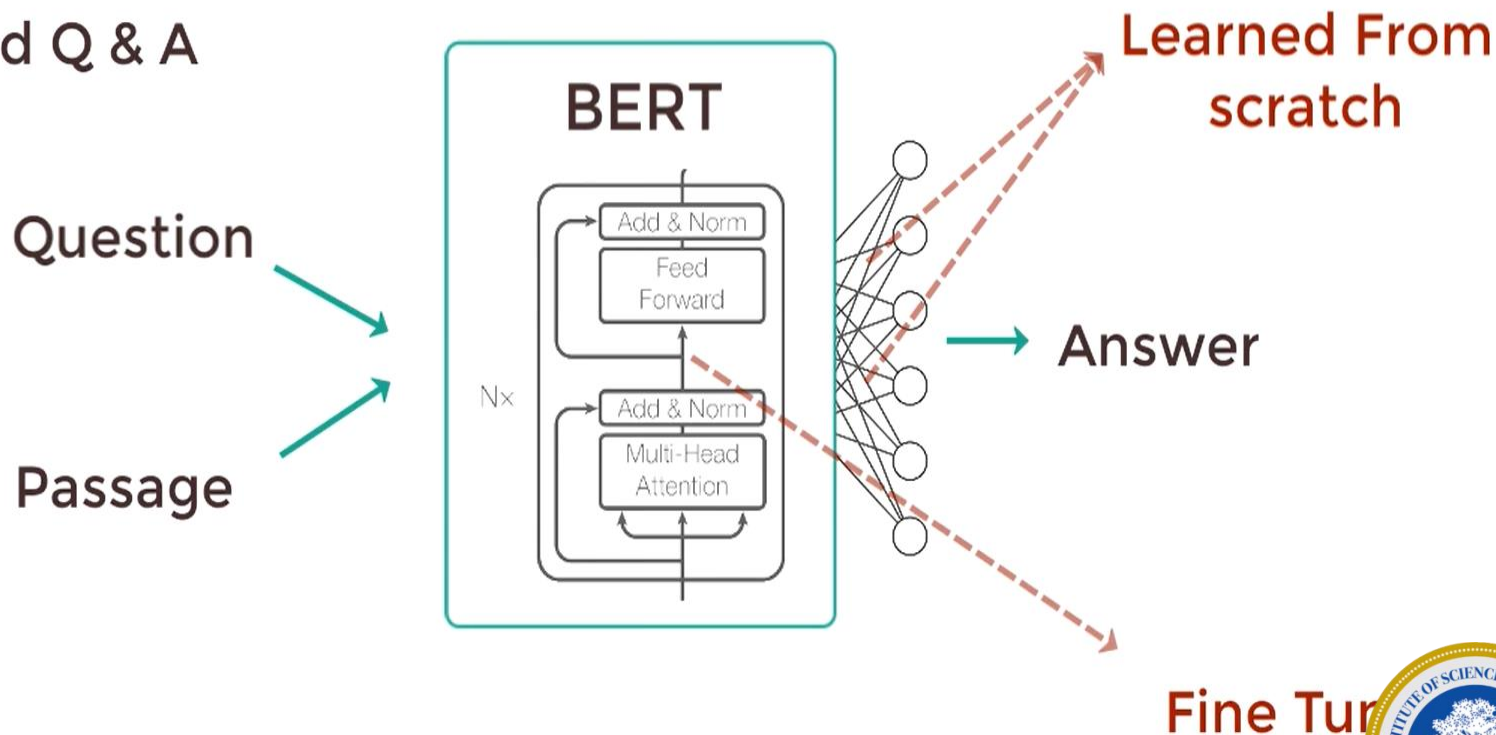
Yes. Sentence B
follows sentence A



Bidirectional Encoder Representation from Transformers

Fine Tuning (Pass 1): “How to use language for specific task?”

Fine tuned Q & A

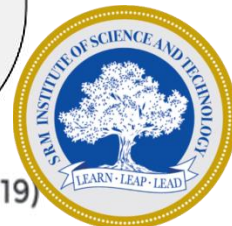
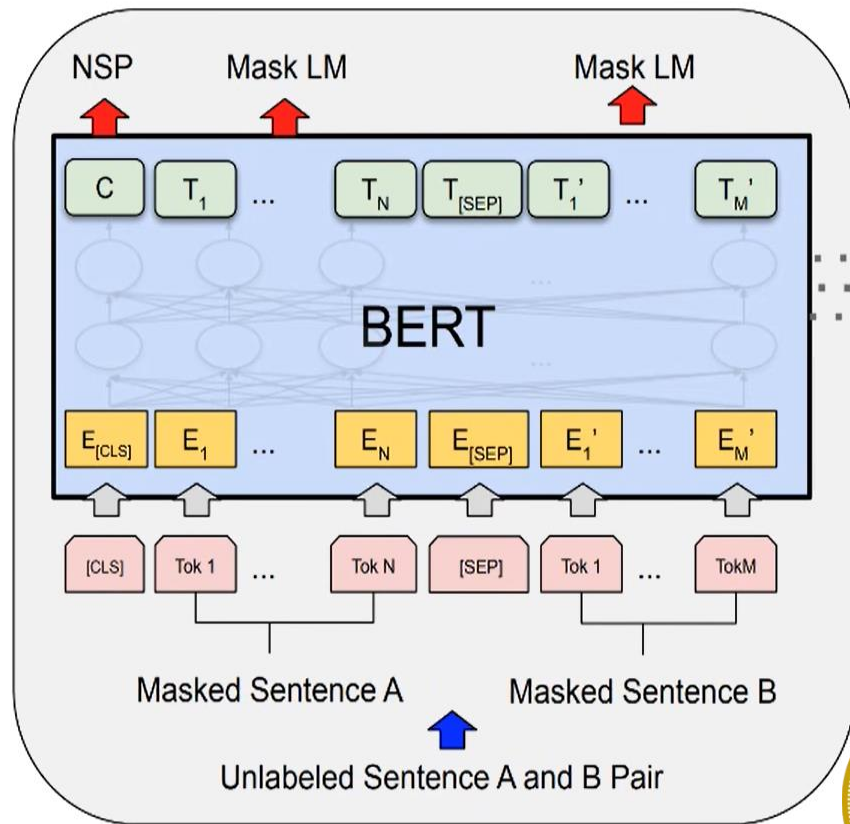


Bidirectional Encoder Representation from Transformers

Pretraining (Pass 2)

Problems to train on
simultaneously:

1. Masked Language Modeling (Mask LM)
2. Next Sentence Prediction (NSP)

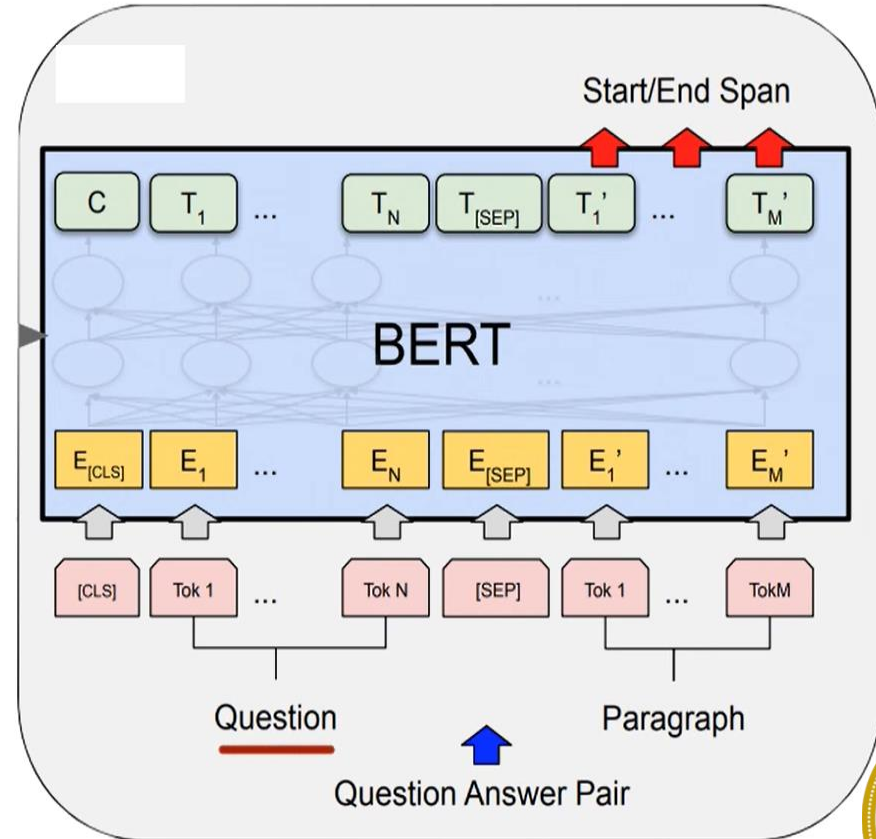


Bidirectional Encoder Representation from Transformers

Fine Tuning (Pass 2)

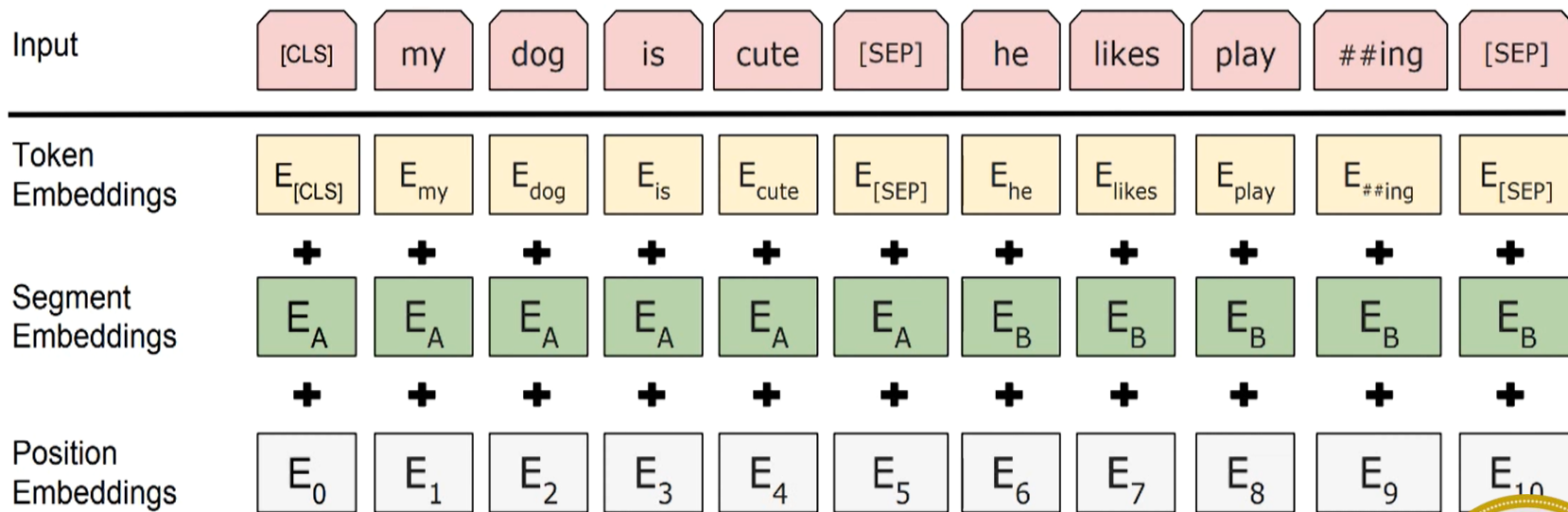
Change output to display
text in which answer exists

Change inputs to take in
Question, Passage



Bidirectional Encoder Representation from Transformers

Pretraining (Pass 3)



Segment + Position embeddings add “ordering” for inputs



BERT

- BERT's input format requires a couple of special tokens to function correctly.
- [CLS]: A token added to the **beginning** of every input sequence. The final hidden state corresponding to this token is used as an aggregate representation for the entire sequence, which is very useful for classification tasks (e.g., sentiment analysis).
- [SEP]: A token used to **separate** sentences. It's used for the NSP task or when you have sentence-pair inputs (e.g., [CLS] a question [SEP] a paragraph [SEP]).
- The output of BERT is a context-rich embedding for every single token in the input.



Word Piece

- **Problem it solves:** Natural language has *tons* of words, plus misspellings, slang, and rare words. A simple word-based vocab would explode in size.
- **Idea:** Start with characters and iteratively merge the most frequent sequences to build *subwords*.
- **Result:**
 - Common words stay whole (play, teacher)
 - Rare/complex words get split (playing → play + ##ing, unhappiness → un + ##happy + ##ness).
- **Special thing:** Uses ## to mark subwords that are *continuations* of a word.
- **Why BERT loves it:**
 - Handles rare words (no more [UNK] everywhere).
 - Keeps vocab small (~30k tokens).
 - Works across multiple languages.

Sentence: "unhappiness"

Tokens: ["un", "##happy", "##ne"]



OG BERT

- **Architecture:** Transformer encoder stack with self-attention; WordPiece tokenizer; max seq length 512.
- **Pretraining tasks:**
 - **MLM (Masked Language Modeling):** predict masked tokens.
 - **NSP (Next Sentence Prediction):** binary task: sentence B follows sentence A?
- **Model sizes:**
 - **BERT-Base:** 12 layers, 768 hidden, 12 heads (~110M params).
 - **BERT-Large:** 24 layers, 1024 hidden, 16 heads (~340M params).
- **Cased vs Uncased:** cased keeps case distinctions; uncased lowercases inputs.
- **[CLS]/[SEP]:** [CLS] pooled output often used for classification; [SEP] separates segments.



RoBERTa

- **Key idea: Train longer, harder, bigger; remove NSP.**
- **Changes vs BERT:**
 - No NSP; dynamic masking; much larger corpora & batch sizes; longer training.
 - Uses byte-level BPE tokenizer (GPT-2 style) instead of WordPiece.
- **Why it helps: NSP was noisy; more data + better masking = stronger encoder representations.**
- **Use when: General-purpose English; strong baseline for most tasks.**



Byte-Pair Encoding (BPE) & Byte-Level BPE

- **Classic BPE:**
 - Start with characters.
 - Iteratively merge the most common *pair* of symbols into a new token.
 - Build a vocab this way.
- **Byte-level BPE** (upgrade used in GPTs):
 - Works directly at the *byte* level (0–255) instead of characters.
 - This means **any text** (Unicode, emojis, code, hashtags, foreign scripts) can be represented without special hacks.
- **Why GPT loves it:**
 - Universal (no language-specific design like WordPiece).
 - No [UNK] tokens, since *every string* is just bytes.
 - Efficient for generative models.

Sentence: "unhappiness"

Might tokenize as: ["un", "happiness"]

or depending on merges: ["un", "hap", "pi",



ALBERT

- **Key idea:** Same compute, way fewer parameters.
- **Changes:**
 - Factorized embeddings (smaller embedding matrix).
 - Cross-layer parameter sharing (weights reused across layers).
- Replaces NSP with SOP (Sentence Order Prediction).
- **Why:** Reduced memory footprint, improved generalization.
- **Use when:** You want BERT-Large-level performance with fewer params (easier to deploy/checkpoint).



ALBERT

- In SOP, the model is given two consecutive sentences from the text:
 - Sometimes in the correct order.
 - Sometimes swapped (order reversed).
- Task: predict if the order is correct or swapped.
- Example:
 - Correct: “He went to the store. He bought some milk.”
 - Swapped: “He bought some milk. He went to the store.”
- SOP forces the model to **understand sentence coherence and logical flow**, rather than just whether two sentences are related at all.

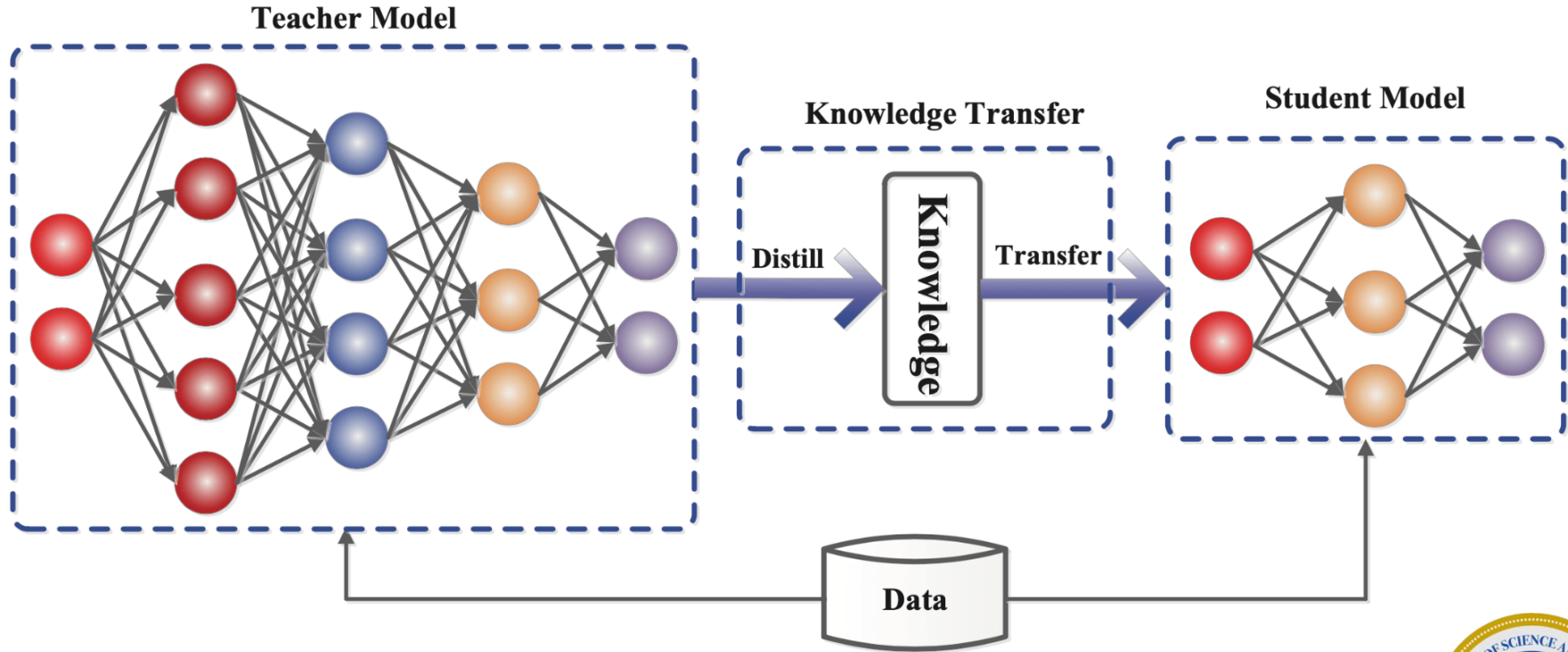


DistilBERT

- **Key idea: Knowledge distillation** of BERT-Base → 6-layer student.
- **Changes:** 40% smaller, ~60% faster, retains ~97% of BERT-Base on GLUE (ballpark).
- **Use when:** Latency/throughput matter, minimal accuracy drop acceptable.



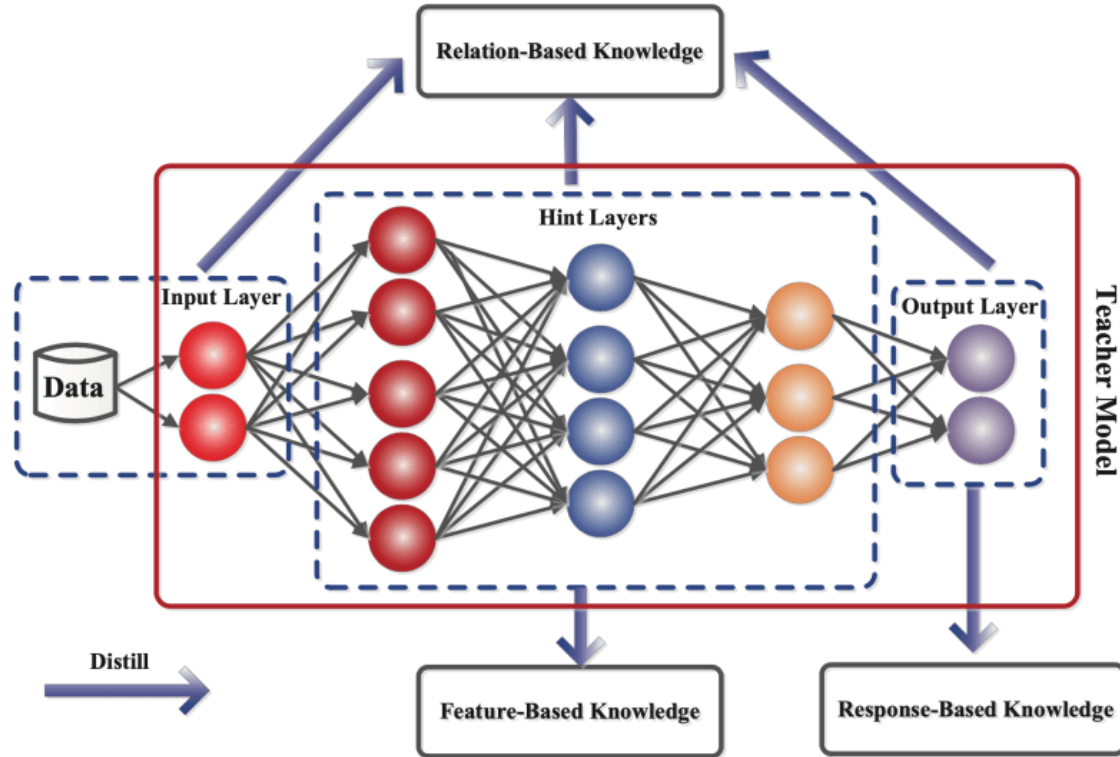
Knowledge Distillation



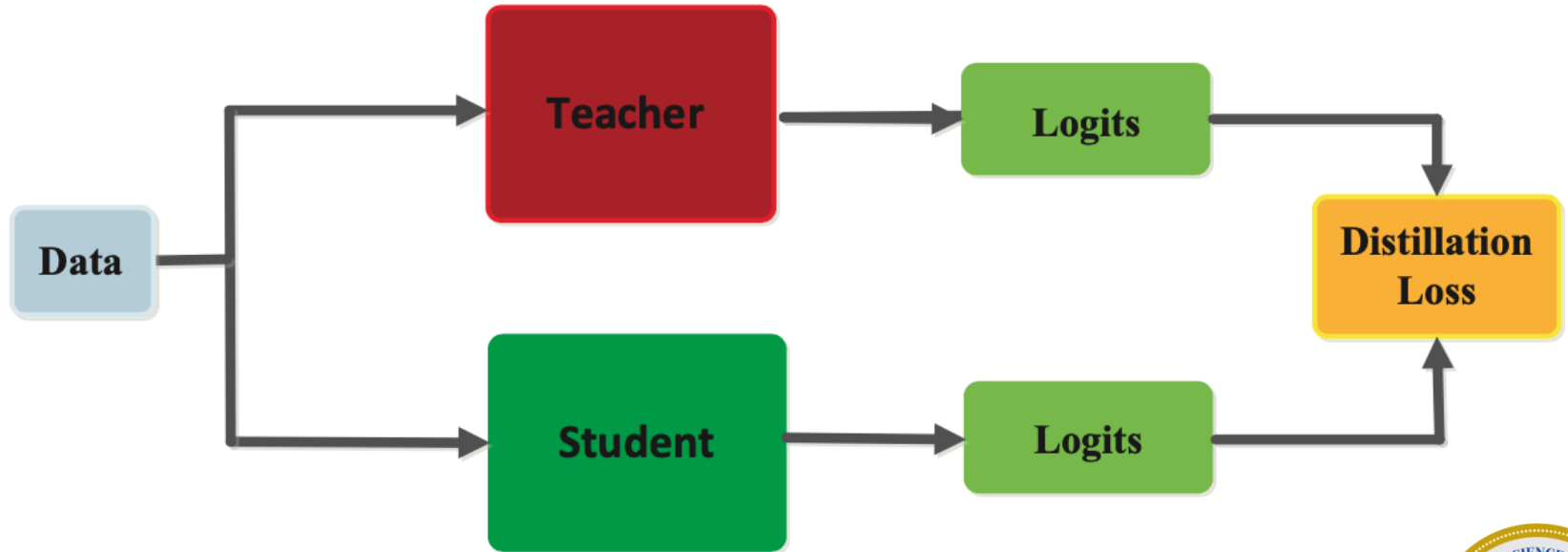
https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/vision/ipynb/knowledge_distillation.ipynb#scrollTo=8fMZMJbCYS0



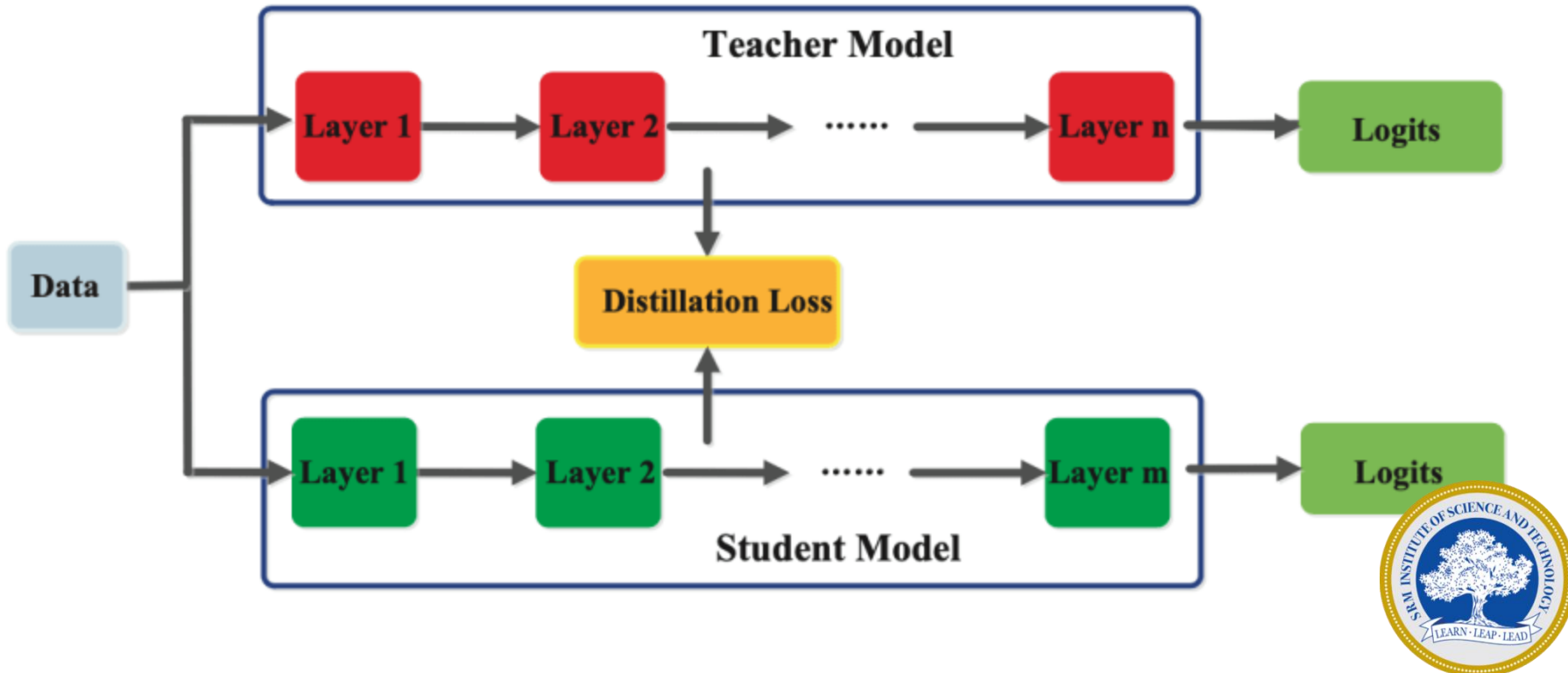
Knowledge Distillation



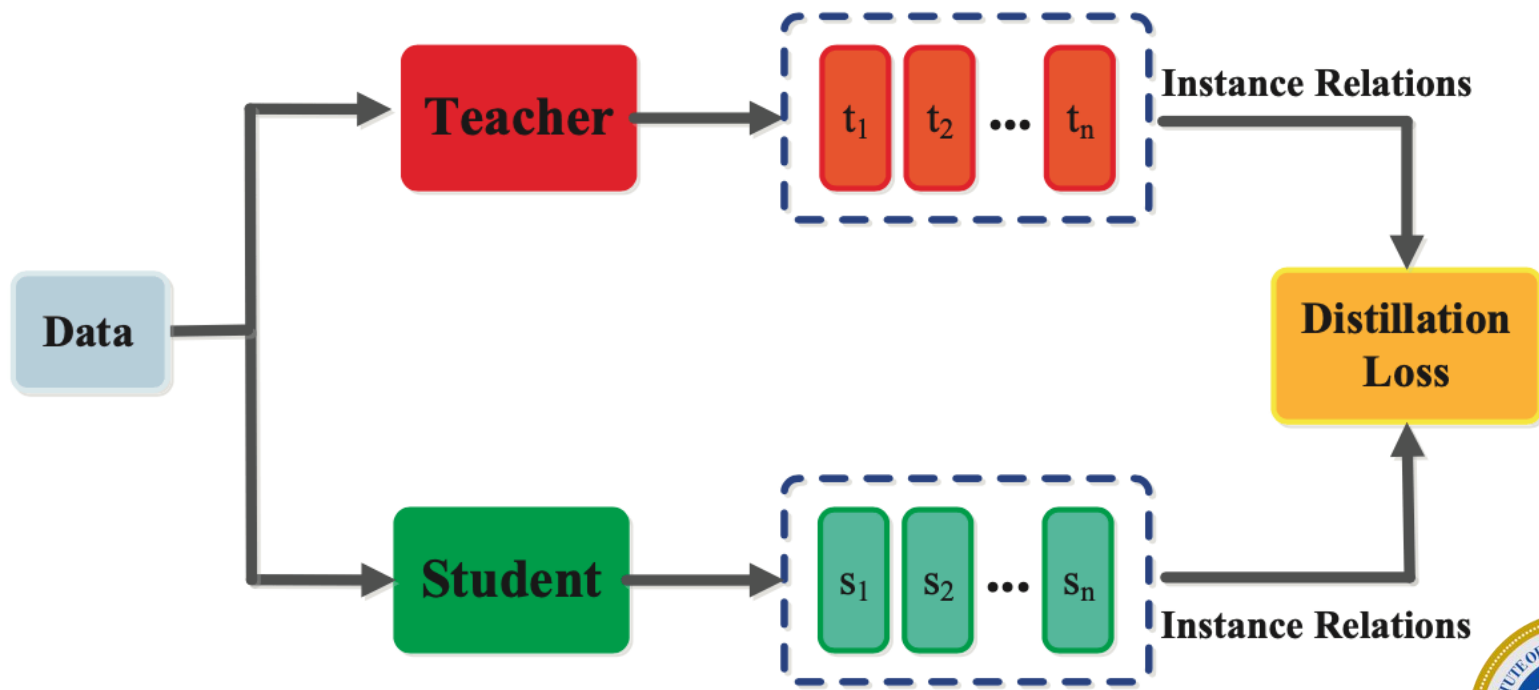
Response-based knowledge



Feature-based knowledge



Relation-based knowledge



Other BERT Variants

- DeBERTa - **Disentangled** content vs position representations; **relative position** biases
- **MobileBERT**: Bottlenecked transformer with teacher-student transfer; tuned for on-device.
- **TinyBERT**: Two-stage distillation (general + task-specific); very small.
- **MiniLM**: Distills **self-attention** maps; excellent **size/perf** trade-off.
- **SqueezeBERT**: Architectural tweaks for speed on commodity hardware.



GPT

- **Architecture:** decoder-only Transformer (causal self-attention; each token can only attend left).
- **Objective:** next-token prediction (autoregressive LM).
- **Tokenizer: byte-level BPE** (handles any Unicode/emoji/code; basically no [UNK]).
- **Strengths:** generation, continuation, few-shot prompting (esp. GPT-3).
- **Weaknesses:** limited context length (512→1024→2048 in these gens), can hallucinate.



GPT 1 - 2018

- **Core idea:** pretrain a decoder-only LM on books, then fine-tune for tasks.
- **Params:** ~117M (12 layers, width ~768, 12 heads).
- **Context window:** ~512 tokens.
- **Data: BooksCorpus** (~7k unpublished books; long, coherent prose).
- **Training:** LM objective; then **supervised fine-tuning** on downstream NLP tasks.
- **Why it mattered:** showed that *one* large LM can be **fine-tuned** to many tasks (classification, NLI, etc.) with strong results vs task-specific models.
- **Limitations:** smaller scale; needed fine-tuning per task; dataset narrow (books only).



GPT-2 (2019)

- **Core idea:** scale up + better data → strong **zero-shot** ability without fine-tuning.
- **Params (released sizes):** 117M, 345M, 762M, **1.5B**.
- **Context window:** **1024** tokens.
- **Data: WebText** (~40GB). Collected from outbound Reddit links with score filters → broader, web-style distribution (news, forums, code snippets).
- **Training:** same LM objective; much longer training + bigger batches; tokenizer = byte-level BPE (~50k vocab).
- **What changed in capability:**
 - **Zero-shot** & few-shot (primitive) performance on translation, QA, summarization just by prompting.
 - Way better **coherence** and **style transfer** vs GPT-1.
- **Limitations:** still needs task setup via prompts; 1024-token cap; safety/bias issues more visible.



GPT-3 (2020)

- **Core idea: massive scale** → **in-context learning** (few-shot via prompt only).
- **Params:** up to **175B** (flagship). (Also smaller: 125M→13B.)
- **Depth/width (175B):** ~96 layers, model dim ~12k, ~96 heads (approx; big).
- **Context window: 2048** tokens.
- **Data (filtered + mixed):** Common Crawl (heavily filtered), WebText2, two large Books sets, Wikipedia → **~300B tokens** total.
- **Training:** same autoregressive LM; lots of regularization + careful optimization (Adam-type, LR schedules, etc.).
- **Breakthrough behavior: in-context learning:**
 - **Zero-shot:** just instructions.
 - **One/few-shot:** include a few examples in the prompt → the model “adapts” without weight updates.
- **Limitations:** 2048-token limit; factuality & bias; prompt-sensitivity; expensive pretrain.



End of Slide

THANK YOU

